

brain_tumor_detector.py

"""

Brain Tumor Detection from MRI Scans using CNN

Author: Mohana Sree K

Dataset: Brain MRI Images for Brain Tumor Detection (Kaggle)

Download: <https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection>

(Also works with the larger 4-class dataset:

<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>)

Expected folder structure after download:

dataset/

yes/ -> MRI images WITH tumor

no/ -> MRI images WITHOUT tumor

(If using the 4-class dataset instead, see the note in the
'Optional: Multi-class version' section at the bottom.)

"""

import os

import numpy as np

import cv2

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.metrics import (

accuracy_score, precision_score, recall_score,

f1_score, confusion_matrix, classification_report

)

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import (

Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization

)

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

from tensorflow.keras.optimizers import Adam

1. Configuration

IMG_SIZE = 128 # resize all images to 128x128

DATASET_DIR = "dataset" # expects dataset/yes and dataset/no

BATCH_SIZE = 32

EPOCHS = 25

RANDOM_STATE = 42

2. Load & Preprocess Images

def load_images(dataset_dir, img_size):

data = []

labels = []

classes = {"no": 0, "yes": 1} # no tumor = 0, tumor = 1

for class_name, label in classes.items():

class_dir = os.path.join(dataset_dir, class_name)

if not os.path.isdir(class_dir):

print(f"Warning: folder not found -> {class_dir}")

continue

for filename in os.listdir(class_dir):

filepath = os.path.join(class_dir, filename)

img = cv2.imread(filepath)

if img is None:

continue

img = cv2.resize(img, (img_size, img_size))

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

```

        data.append(img)
        labels.append(label)

    data = np.array(data, dtype="float32") / 255.0    # normalize pixel values
    labels = np.array(labels)
    return data, labels

print("Loading and preprocessing images...")
X, y = load_images(DATASET_DIR, IMG_SIZE)
print(f"Total images loaded: {len(X)}")
print(f"Tumor cases: {np.sum(y == 1)} | Non-tumor cases: {np.sum(y == 0)}")

# -----
# 3. Train/Val/Test Split
# -----
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=RANDOM_STATE, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=RANDOM_STATE, stratify=y_temp
)

print(f"Train: {len(X_train)} | Val: {len(X_val)} | Test: {len(X_test)}")

# -----
# 4. Data Augmentation
# -----
# Augmentation helps generalization on a limited medical imaging dataset
train_datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode="nearest"
)
train_datagen.fit(X_train)

# -----
# 5. Build CNN Model
# -----
def build_model(input_shape):
    model = Sequential([
        Conv2D(32, (3, 3), activation="relu", padding="same", input_shape=input_shape),
        BatchNormalization(),
        MaxPooling2D(2, 2),

        Conv2D(64, (3, 3), activation="relu", padding="same"),
        BatchNormalization(),
        MaxPooling2D(2, 2),

        Conv2D(128, (3, 3), activation="relu", padding="same"),
        BatchNormalization(),
        MaxPooling2D(2, 2),

        Conv2D(128, (3, 3), activation="relu", padding="same"),
        BatchNormalization(),
        MaxPooling2D(2, 2),

        Flatten(),
        Dense(256, activation="relu"),
        Dropout(0.5),
        Dense(64, activation="relu"),
        Dropout(0.3),
        Dense(1, activation="sigmoid")    # binary classification: tumor / no tumor
    ])

    model.compile(
        optimizer=Adam(learning_rate=0.0001),

```

```

        loss="binary_crossentropy",
        metrics=["accuracy", tf.keras.metrics.Precision(name="precision"),
                 tf.keras.metrics.Recall(name="recall")]
    )
    return model

model = build_model((IMG_SIZE, IMG_SIZE, 3))
model.summary()

# -----
# 6. Train the Model
# -----
callbacks = [
    EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True),
    ModelCheckpoint("best_brain_tumor_model.h5", monitor="val_loss", save_best_only=True)
]

history = model.fit(
    train_datagen.flow(X_train, y_train, batch_size=BATCH_SIZE),
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    callbacks=callbacks
)

# -----
# 7. Evaluate on Test Set
# -----
y_pred_probs = model.predict(X_test)
y_pred = (y_pred_probs > 0.5).astype(int).flatten()

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\n--- Test Set Performance ---")
print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f} (critical: minimizes false negatives)")
print(f"F1 Score: {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["No Tumor", "Tumor"]))

# -----
# 8. Confusion Matrix
# -----
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No Tumor", "Tumor"], yticklabels=["No Tumor", "Tumor"])
plt.title("Confusion Matrix - Brain Tumor Detection")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.tight_layout()
plt.savefig("confusion_matrix.png")
print("\nConfusion matrix saved as confusion_matrix.png")

# -----
# 9. Training Curves
# -----
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train Accuracy")
plt.plot(history.history["val_accuracy"], label="Val Accuracy")
plt.title("Accuracy over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

```

```

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Val Loss")
plt.title("Loss over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.savefig("training_curves.png")
print("Training curves saved as training_curves.png")

# -----
# 10. Save Final Model
# -----
model.save("brain_tumor_cnn_model.h5")
print("\nFinal model saved as brain_tumor_cnn_model.h5")

# -----
# 11. Predict on a New MRI Image
# -----
def predict_tumor(image_path, model, img_size=IMG_SIZE):
    img = cv2.imread(image_path)
    img = cv2.resize(img, (img_size, img_size))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = img.astype("float32") / 255.0
    img = np.expand_dims(img, axis=0)

    prob = model.predict(img)[0][0]
    label = "Tumor Detected" if prob > 0.5 else "No Tumor Detected"
    print(f"Prediction: {label} (confidence: {prob:.2%})")
    return label, prob

# Example usage (uncomment and provide a real path to test):
# predict_tumor("dataset/yes/Y1.jpg", model)

```